

***DiskOnChip Driver Extended Functions
Based on TrueFFS 6.x
Developer Guide***

**December 2003
91-SR-006-11-7L Rev. 1.1**

DOCUMENT CONTROL INFORMATION

DCO No.:

	Title	Name	Date
Issued by:	DiskOnChip Product Manager	Amir Fridman	December 25, 2003
Approved by:	VP Operations and Product Management	Rami Koren	December 25, 2003

TABLE OF CONTENTS

1. TrueFFS.....	5
1.1. Introduction	5
1.2. Terms and Definitions	5
2. Extended Functionality Interface.....	8
3. Extended Function Argument Structures.....	11
3.1. FL_IOCTL_GET_INFO.....	12
3.2. FL_IOCTL_DEFRAGMENT	14
3.3. FL_IOCTL_WRITE_PROTECT	15
3.4. FL_IOCTL_MOUNT_VOLUME	16
3.5. FL_IOCTL_FORMAT_VOLUME	17
3.6. FL_IOCTL_DELETE_SECTORS	19
3.7. FL_IOCTL_FORMAT_PHYSICAL_DRIVE.....	20
3.7.1. Translation Layer Formatting.....	21
3.7.2. Special Format Features	22
3.7.3. Additional Special Features	23
3.7.4. BDTL Partition Parameters.....	23
3.7.5. Binary Partition Parameters.....	25
3.7.6. Standard Formatting Parameters	25
3.8. Hardware Protection.....	28
3.8.1. Overview	28
3.8.2. Method of Operation	28
3.8.3. FL_IOCTL_BDTL_HW_PROTECTION	30
3.8.4. FL_IOCTL_BINARY_HW_PROTECTION.....	31
3.9. FL_IOCTL_OTP	32
3.10. Unique ID (UID).....	34
3.10.1. FL_IOCTL_CUSTOMER_ID.....	34
3.10.2. FL_IOCTL_UNIQUE_ID	34
3.11. FL_IOCTL_NUMBER_OF_PARTITIONS	35
3.12. FL_IOCTL_INQUIRE_CAPABILITIES	36
3.13. FL_IOCTL_SET_ENVIRONMENT_VARIABLES	37
3.14. FL_IOCTL_PLACE_EXB_BY_BUFFER	38
3.15. FL_IOCTL_WRITE_IPL.....	39
3.16. FL_IOCTL_DEEP_POWER_DOWN_MODE	40

3.17. FL_IOCTL_BDK_OPERATION	41
3.17.1. BDK_INIT_READ.....	42
3.17.2. BDK_READ.....	43
3.17.3. BDK_INIT_WRITE	43
3.17.4. BDK_WRITE	43
3.17.5. BDK_ERASE	44
3.17.6. BDK_CREATE.....	44
3.17.7. BDK_GET_INFO	45
3.18. FL_IOCTL_EXTENDED_ENVIRONMENT_VARIABLES	46
3.19. FL_IOCTL_VERIFY_VOLUME	50
3.20. FL_IOCTL_EXTENDED_WRITE_IPL	51
3.21. FL_IOCTL_IS_QUICK_MOUNT_VALID	52
3.22. FL_IOCTL_WRITE_QUICK_MOUNT_INFO.....	52
3.23. FL_IOCTL_CLEAR_QUICK_MOUNT_INFO	53
3.24. FL_IOCTL_COMPLETE_OPERATION.....	54
4. Additional Information.....	55
How to Contact Us	56

1. TRUEFFS

1.1. Introduction

The basic function of TrueFFS is to enable hard-disk emulation using DiskOnChip. To do this, TrueFFS provides a standard block-device interface capable of reading from and writing to logical sectors. This capability enables file-systems and operating systems to access DiskOnChip as a storage device.

In addition to standard data storage device functionality, the TrueFFS driver (based on the TrueFFS SDK) provides extended functionalities, including features such as: formatting the media, read/write protection, accessing binary partitions, and flash defragmentation. These unique functionalities are available in most TrueFFS-based drivers through the standard I/O control command of the operating system (OS).

In many OSs, the I/O Control (IOCTL) mechanism is the only way to access a driver running in kernel mode from applications running in user mode.

All TrueFFS-based drivers have the same internal extended functionalities mechanism. However, various OS constraints force several variations to the external interface. This document describes the core interface on which specific variations must be made for individual OSs. Examples of using the extended functions are included with the installation manuals (or readme files) of the OS-specific drivers or in source code examples.

1.2. Terms and Definitions

Table 1: List of Terms

Interface Term	Definition
Socket	The physical location where a DiskOnChip device can reside.
MTD	Memory Technology Driver. A software module that abstracts the physical flash medium and handles all flash read, write and erase operations.
SLC	Single Level Cell. Flash technology that stores one bit per flash cell.
MLC	Multi Level Cell. Flash technology that stores more than one bit per flash cell.
Planes	Some flash devices support flash programming to several flash pages in a single flash command, therefore achieving parallelism in time-consuming flash program-and-erase operations.
Erase unit or Block	Smallest erasable area of the flash using the MTD.
Flash Page	The smallest area of the flash (usually 512 bytes long) written in a single flash write command

Interface Term	Definition
XIP	eXecute In Place. Memory regions that may contain executable code, enabling the CPU to run directly from those regions. Note: Normal NAND memory does not have XIP functionality, and requires special code (i.e. the BDK) to read from the flash to an XIP region, such as the system RAM.
IPL	Initial Program Loader. DiskOnChip can initialize a segment of the 8KB memory window with a small XIP piece of code. A dedicated area on the flash is usually loaded to DiskOnChip's internal static RAM. This code can be used for system initialization. For systems that need a larger initialization code, the IPL can be used to load an SPL from the flash to the system RAM.
SPL	Secondary Program Loader. The DiskOnChip XIP area is usually limited to a few KB in size, and cannot accommodate the ECC (the EDC is done in hardware). This limitation prevents the IPL from loading the full OS image. Instead, the IPL is used to initialize the system RAM and load the SPL from the flash. The SPL usually comprises a few simple calls to the BDK, allowing you to load any amount of code (while skipping bad blocks), provided the device has sufficient capacity.
Firmware Space \ EXB Space	M-Systems provides a firmware file for all products that contain an IPL, SPL and a driver for x86 BIOS platforms. This file has an .EXB extension.
Partition / Volume	Part of a physical drive that is handled as an independent unit. A partition can be either a disk partition (logical drive partition) or a binary partition.
Disk Partition (Logical Drive / BDTL Partition / BDTL Volume)	A partition formatted and supported by one of the TrueFFS Translation Layers (NFTL, INFTL or SAFTL), making it capable of supporting a block device driver and file system.
Binary Partition \ Binary Volume	A partition that is not supported by the TrueFFS block device translation layer (BDTL), but which can be used as a direct-access storage area (skipping bad blocks). Binary partitions are numbered starting from zero, independently of the disk partitions co-existing on the same physical drive. The maximum number of binary partitions is set by the BINARY_PARTITIONS parameter defined in FLCUSTOM.H.

Interface Term	Definition
Binary Sub-Partition	All binary partition blocks are marked with a unique signature. Starting from TrueFFS-SDK (OSAK) 4.1, a dedicated routine enables changing this signature for a contiguous subset of the partition blocks, creating several separate areas within the binary partition. Each area is called a binary sub-partition. When first formatted, the binary partition contains a single sub-partition.
SPL Partition	A simplified binary partition that does not use the matching algorithm, making it ideal for storing the SPL code. The simpler logic simplifies the code required for reading, and allows it to fit on the DiskOnChip XIP block.
INFTL-Formatted Device	Inverse NAND Flash Translation Layer. The flash management algorithm used by TrueFFS to manage DiskOnChip 2000, Mobile DiskOnChip, and DiskOnChip Millennium Plus products.
NFTL-Formatted Device	NAND Flash Translation Layer. The flash management algorithm used by TrueFFS to manage DiskOnChip 2000 and DiskOnChip Millennium products.
SAFTL-Formatted Device	Sequential Access Flash Translation Layer. The flash management algorithm used by TrueFFS to manage DiskOnChip G3/P3 and DiskOnChip 2000 G3 products.

2. EXTENDED FUNCTIONALITY INTERFACE

The TrueFFS driver implements the extended functionalities interface through OS device I/O control calls. The device I/O control interface may vary between OSs, but the general interface requires the following two steps:

1. Get the TrueFFS driver descriptor:

```
Drive Handle = Operating System Get Drive Handle Call (Device Driver
Name & Parameters);
```

2. Initiate an extended function call:

```
File System Call Status = File System Io Control Call ( Driver
Handle , I/O Request Packet );
```

Step 1 is file-system dependent. Its result is a descriptor of the TrueFFS driver that can be used by the device I/O control calls. Step 2 has several attributes common to all file systems:

- **File System Call Status:** This value indicates whether the call was successfully passed to the driver and whether the driver took responsibility for the call. It usually does not specify the operation status. The operation status is returned as part of the I/O request packet using standard TrueFFS status codes. The complete set of the TrueFFS status codes is available in the I/O control H files, included with each driver package.
- **File System I/O Control Call:** The function name used to invoke a file system extended functionality (device I/O control) call.
- **I/O Request Packet:** All TrueFFS extended functions receive the following I/O request packet:

```
typedef struct {
    FLHandle          irHandle;
    unsigned          irFlags;
    void FAR1         * irPath;          /* Not used */
    void FAR1         * irData;
    FLSDword          irLength;          /* Not used */
    FLSDword          irCount;           /* Not used */
} Ioreq
```

- **irHandle:** A handle identifying the partition on which an operation should be performed. When there is only one logical drive, no ambiguity can occur and the drive handle parameter should be 0. This drive handle is composed of the physical drive number (LSB 0-3) and the partition number (LSB 4-7).

Note: As both binary partitions and disk partitions are numbered from 0, a binary partition can have the same handle as a disk partition on the same physical drive. Therefore, binary and disk operations are always handled using different calls to avoid ambiguity.

- **irFlags:** This field controls the type of extended function. Every extended function is represented by a code defined as enumerated type.

```
typedef enum{ FL_IOCTL_GET_INFO = FL_IOCTL_START,
FL_IOCTL_DEFRAGMENT,
FL_IOCTL_WRITE_PROTECT,
FL_IOCTL_MOUNT_VOLUME,
FL_IOCTL_FORMAT_VOLUME,
FL_IOCTL_BDK_OPERATION,
FL_IOCTL_DELETE_SECTORS,
FL_IOCTL_READ_SECTORS,
FL_IOCTL_WRITE_SECTORS,
FL_IOCTL_FORMAT_PHYSICAL_DRIVE,
FL_IOCTL_FORMAT_LOGICAL_DRIVE,          /* Not implemented */
FL_IOCTL_BDTL_HW_PROTECTION,
FL_IOCTL_BINARY_HW_PROTECTION,
FL_IOCTL_OTP,
FL_IOCTL_CUSTOMER_ID,
FL_IOCTL_UNIQUE_ID,
FL_IOCTL_NUMBER_OF_PARTITIONS,
FL_IOCTL_INQUIRE_CAPABILITIES,
FL_IOCTL_SET_ENVIRONMENT_VARIABLES,     /* Not supported */
FL_IOCTL_PLACE_EXB_BY_BUFFER,
FL_IOCTL_WRITE_IPL,                     /* Not supported */
FL_IOCTL_DEEP_POWER_DOWN_MODE,
FL_IOCTL_EXTENDED_ENVIRONMENT_VARIABLES,
FL_IOCTL_VERIFY_VOLUME,
FL_IOCTL_SET_ACCESS_ROUTINE,
FL_IOCTL_GET_ACCESS_ROUTINE,
FL_IOCTL_EXTENDED_WRITE_IPL,
FL_IOCTL_OPEN_TRANSACTION,              /* Not supported */,
FL_IOCTL_COMMIT_TRANSACTION,            /* Not supported */,
FL_IOCTL_IS_VOLUME_RUGGEDIZED,          /* Not supported */,
FL_IOCTL_IS_QUICK_MOUNT_VALID,
FL_IOCTL_WRITE_QUICK_MOUNT_INFO,
```

```
FL_IOCTL_CLEAR_QUICK_MOUNT_INFO,  
FL_IOCTL_COMPLETE_OPERATION  
} fl_IOCTLFunctionNo;
```

The constant **FL_IOCTL_START** defines the number of the first extended function code to be used by TrueFFS. Typically, each OS defines a range of extended function codes that are reserved for its own use. **FL_IOCTL_START** is therefore defined outside this range (see the OS driver installation manual for more information).

- **irData:** This field should contain a pointer to an **fl_IOCTLRecord** record. The **fl_IOCTLRecord** record contains pointers to your specific extended function input and output records:

```
typedef struct {  
    void FAR1 *inputRecord;  
    void FAR1 *outputRecord;  
} fl_IOCTLRecord;
```

Section 3 describes the specific input and output records of each extended function in detail.

Calling extended functions depends on the OS, and is described in detail in the relevant manual of each specific TrueFFS driver or provided as a sample source code application.

3. EXTENDED FUNCTION ARGUMENT STRUCTURES

This section describes the purpose of each extended function, along with usage and relevant I/O structures. As explained in Section 2, all extended functions receive a common **flIOctlRecord** pointing to the respective input and output records:

```
typedef struct {  
    void FAR1 *inputRecord;  
    void FAR1 *outputRecord;  
} flIOctlRecord;
```

The status of the I/O control call is returned in the output records of the TrueFFS driver (**FLStatus** field). **FLOK (0)** indicates success, while any other status indicates some kind of failure.

Note: The file system call status indicates whether the call was successfully passed to the driver and whether the driver took responsibility for the call. It does not specify the operation status. The operation status is one of the **outputRecord** fields using standard TrueFFS status codes.

3.1. FL_IOCTL_GET_INFO

This function returns general information on the specific disk (BDTL) partition, the DiskOnChip socket address, software version, high-level and low-level geometry and estimated lifetime of the media.

A **VolumeInfoRecord** structure is returned to a user buffer containing the information. The **VolumeInfoRecord** structure is defined as:

```
typedef struct {
    FLDword logicalSectors;
    FLDword bootAreaSize;      /* boot area size of the entire drive
                               (combines all binary partitions) */
    FLDword baseAddress; /* physical base address of the memory window */
    FLWord  flashType;      /* JEDEC id of the flash          */
    FLDword physicalSize;    /* physical size of the media in bytes */
    FLDword physicalUnitSize; /* flash erasable block size in bytes */
    FLSByte DOCTYPE;         /* DiskOnChip types */
    FL_NOT_DOC                - 0   - Not DiskOnChip
    FL_DOC                    - 1   - DiskOnChip 2000
    FL_MDOC                   - 2   - DiskOnChip Millennium products
    FL_MDOCP_16               - 4   - DiskOnChip Millennium Plus and
    Mobile DiskOnChip 16MB(128Mb) and 32MB(256Mb) dual-die products
    (interleave-1)
    FL_MDOCP                   - 5   - DiskOnChip Millennium Plus and
    Mobile DiskOnChip 32MB(256Mb) single die and 64MB(512Mb) products
    (interleave-2)
    FL_DOC_OREN               - 6   - DiskOnChip 2000 G3 products
    FL_MDOC512_G3             - 7   - DiskOnChip G3 products
    FL_MDOC256_P3             - 8   - DiskOnChip P3 products

    FLSByte lifetime; /* lifetime indicator for the partition (1-10) */
    /* 1   - the media is fresh                                */
    /* 10  - the media is close to the end of its life          */
    FLSByte driverVer[10]; /* driver version (NULL terminated string) */
    FLSByte OSAKVer[10];  /* TrueFFS version that driver is based on
                           (NULL terminated string) */

    /* The following values are of the specific partition */
    FLDword cylinders; /* Media..... */
    FLDword heads;     /* geometry..... */
}
```

```

FLDword sectors;      /* parameters.....          */
} VolumeInfoRecord;    /* end struct */

```

Table 2: VolumeInfoRecord Parameters

Parameter	Definition
logicalSectors	Number of logical sectors.
bootAreaSize	Number of physical bytes (this amount may include unusable bad blocks) reserved for the binary partition.
baseAddress	Physical address in the host memory where the DiskOnChip window is located.
physicalSize	Amount of raw flash memory available, in bytes. The actual amount of storage space available for data storage is lower due to formatting overhead and presence of a binary partition.
physicalUnitSize	Size of erasable flash blocks, in bytes. Note: This field was changed to unsigned long from unsigned short.
DOCType	Family of products to which this DiskOnChip device belongs: DiskOnChip 2000 (DIP or DIMM), DiskOnChip Millennium, DiskOnChip Millennium Plus, or Mobile DiskOnChip.
lifetime	Since DiskOnChip is flash memory, it is limited by the possible number of erase cycles. This parameter indicates the "lifetime status" (1 through 10), where: 1 : Indicates the media is fresh 10 : Indicates that the media is close to its end of life Note: This value is only an estimate based on general lifetime statistics.
driverVer	Version number of the TrueFFS driver for this specific OS. Note: this is not the TrueFFS SDK version number.
OSAKVer	TrueFFS SDK version on which the TrueFFS driver is based.
cylinders	Media geometry parameter: number of cylinders.
heads	Media geometry parameter: number of heads
sectors	Media geometry parameter: number of sectors per track.

Input Record

DO NOT CARE.

Output Record

```

typedef struct {
    VolumeInfoRecord info;
    FLStatus status;} flDiskInfoOutput;

```

3.2. FL_IOCTL_DEFRAGMENT

Performing regular write operations to the flash makes periodic space reclamation, or garbage collection, necessary. TrueFFS performs garbage collection automatically, usually on an immediate-need basis. This process takes time and slows down both the average and maximum time for performing a write operation.

The defragmentation process performs early garbage collection. If an application needs to write a burst of data and has some idle time before the burst arrives, it can write the data more quickly when applying an early defragmentation call. If it is necessary to write data without interruption for garbage collection, defragmentation should be done before starting the write operation. The minimum number of bytes specified should include about 20% extra for FAT and BDTL overhead. For example, to write a 16KB file, specify about 40 sectors as a defragmentation target.

This function accepts the minimum number of sectors required for an immediate write operation as a parameter. If the current amount of available space is greater than this parameter, the function returns immediately. If not, garbage collection is performed until the amount of free space is at least equal to the quantity requested or until there is no reclaimable space left on the media.

If the amount of required free space is unknown, a quick garbage collection procedure can be invoked by setting the required number of sectors to -1. The amount of space reclaimed during this operation depends on the physical geometry of the media, and the distribution of the data on the media. However, garbage collection is performed in the most efficient manner (that is, the best “space reclaimed to time of operation” ratio). In all cases, the call returns the actual number of sectors currently available for writing.

Note: This number is **not** the same as the free space on the volume, but represents only the amount of flash memory in the erased state. A volume may be empty of files, yet have no sectors available for writing.

To determine the amount of space currently available, request a defragmentation for 0 (zero) sectors. Defragmentation is not performed, however, the current number of free sectors is returned. To perform a general defragmentation of the volume, request a large number of sectors. In this case, defragmentation completes with a failing status, since the number requested cannot be achieved.

Input Record

```
typedef struct { /* Minimum number of sectors to make available */
    FLSDword requiredNoOfSectors; /* -1 will invoke a quick garbage
    collection */
} flDefragInput;
```

Output Record

```
typedef struct {
    FLSDword actualNoOfSectors; /* Actual number of sectors available */
    FLStatus status;
} flDefragOutput;
```

3.3. FL_IOCTL_WRITE_PROTECT

This function enables key-controlled software write protection for a disk partition. Once a disk partition is protected by the key, it remains in read-only mode. Removing a key can be done by an authorized user who knows the current key.

The key consists of 8 bytes (64 bits), each of which may be any 8-bit code character (2^{64} combinations). The key is stored on the flash disk in a manner that is both scrambled and hidden. That is, the key is encrypted, and it is not possible to read the flash disk to see the encrypted key. If the key is lost or forgotten by the authorized user, the flash disk can be restored to read/write mode by downloading all data from it, reformatting it, and uploading the saved data. A new key can then be enforced.

The same procedure can also be performed by unauthorized users. In this case however, the authorized user is able to determine that the key was removed or changed.

A key-protected DiskOnChip is available to unauthorized users in read-only mode. All data may be read, but not written or modified. An authorized user can write to the flash disk by temporarily disabling the write protection (unlock) or permanently removing it (unprotect), depending on the parameters involved. If the protection is temporarily removed, dismounting DiskOnChip and/or performing a system reset causes DiskOnChip to revert to read-only mode.

DiskOnChip units are not key-protected by default when shipped by M-Systems.

Note: This protection is not as reliable as the hardware protection supported by DiskOnChip Millennium Plus, Mobile DiskOnChip, DiskOnChip 2000 G3 and DiskOnChip G3 products.

Input Record

```
typedef struct {
    FLByte type;          /* Type of operation:                      */
                          /* FL_PROTECT / FL_UNPROTECT / FL_UNLOCK      */
    FLSDword password[2]; /* 8 bytes Key                                */
} flWriteProtectInput

#define FL_PROTECT      0 - Make the DiskOnChip write-protected.
#define FL_UNPROTECT    1 - Permanently remove the write-protection.
#define FL_UNLOCK       2 - Temporarily remove the write-protection.
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

3.4. FL_IOCTL_MOUNT_VOLUME

This function remounts DiskOnChip. Remounting consists of discarding all in-memory control information kept by the TrueFFS driver, and rebuilding it. The remount consists of a low-level BDTL mount.

This function is only required under special circumstances. One of the most common uses of this function is when a user application accesses and modifies DiskOnChip via a method other than the file system or the TrueFFS driver API, such as using the standalone DFORMAT DiskOnChip formatting utility. In this case, the TrueFFS driver is not updated with the changes to DiskOnChip, and every operation becomes unreliable or even harmful. Forcing TrueFFS to remount the driver causes it to get updated with the changes, and enables it to operate reliably.

Input Record

```
typedef struct {
    FLByte type;
} flMountInput;

#define FL_MOUNT          0
#define FL_DISMOUNT      1
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

Note: Invalid arguments will force remounting DiskOnChip (identical to **FL_MOUNT**).

3.5. FL_IOCTL_FORMAT_VOLUME

This function formats a volume, writing a new and empty file system, and all existing data is destroyed. Optionally, a low-level (flash translation layer) formatting can be performed.

FL_IOCTL_FORMAT_VOLUME is included only for backwards compatibility with previous versions of the TrueFFS SDK. Whenever possible, use **FL_IOCTL_FORMAT_PHYSICAL_DRIVE** instead.

Input Record

```
typedef struct {
    FLByte formatType;           /* Type of format          */
    formatParams fp;             /* Format parameters structure */
} flFormatInput;
```

Options for formatType:

```
#define FAT_ONLY_FORMAT          0 -
Perform FAT formatting only without the low-level format.

#define TL_FORMAT                1 -
Perform both low-level and FAT format.

#define TL_FORMAT_IF_NEEDED      2 -
Perform low-level and FAT format only if the current FAT format
is invalid.

#define TL_FORMAT_ONLY           8 -
Perform a low-level format only.
```

The **FormatParams** structure is defined in **FLFORMAT.H** with the following fields:

<pre>FLSDword bootImageLen;</pre>	Space to reserve for a boot image at the start of the medium. The disk partition starts at the next higher erase-unit boundary. A boot image can contain binary data. Binary data will not be moved around on the media, even during garbage collection. To reformat a previously formatted DiskOnChip device while leaving the old boot image, assign -1 to bootImageLen.
<pre>unsigned percentUse;</pre>	NAND flash inherently contains some bad blocks. TrueFFS handles those blocks by managing a pool of spare blocks also called transfer units. This format parameter specifies the percentage of the media to use. According to the flash specifications, SLC flashes should use 98% while MLC flashes should use 95%.

```
unsigned noOfSpareUnits;
```

```
FLDword vmAddressingLimit;
```

```
FLProgressCallback  
progressCallback;
```

```
FLSByte volumeId[4];
```

```
FLSByte FAR1 * volumeLabel;
```

```
unsigned noOfFATcopies;
```

```
unsigned embeddedCISlength;
```

```
FLSByte FAR1 * embeddedCIS;
```

Output Record

```
typedef struct {  
    FLStatus status;  
} flOutputStatusRecord;
```

You can let TrueFFS automatically choose the percentage according to the flash spec by using the **TL_DEFAULT_PERCENTAGE** definition.

A disk partition needs at least one spare erase unit to function as read/write media. It is possible to specify more than one spare unit, which uses more media space. This ensures that if one of the flash erase units becomes bad and inerasable, then one of the spare units can replace it, enabling TrueFFS to continue its read/write functionality. Conversely, if no second spare unit is available the media may switch into read-only mode. The standard value is 3.

Reserved for non-DiskOnChip devices (NOR flash).

Pointer to user callback routine.

FLStatus (*progressCallback)

(FLWord totalUnitsToFormat, FLWord
totalUnitsFormattedSoFar);

Volume identification number.

Volume label string. If NULL, no label.

It is customary to format DOS media with two FAT copies. The first copy is always used, but more copies make it possible to recover the FAT if becomes corrupted (a rare occurrence). Conversely, this slows down performance and uses media space. The standard value is 2.

Reserved for non-DiskOnChip devices (NOR flash).

Reserved for non-DiskOnChip devices (NOR flash).

3.6. FL_IOCTL_DELETE_SECTORS

This function marks one or more consecutive absolute sectors as logically deleted, and is required only under special circumstances. TrueFFS write performance depends to some degree on the amount of free space on the flash disk. A flash disk that is close to being full will show slower write performance, as garbage collection must be performed more often.

FL_IOCTL_DELETE_SECTORS is used to increase the amount of logically free space by informing TrueFFS of absolute sectors that it considers used, but can be deleted as they no longer contain useful data.

Typically, **FL_IOCTL_DELETE_SECTORS** is beneficial for systems that use a non-FAT file system. If the file-system code is available, it is possible to identify the places where the file system marks disk areas as logically deleted, and to inform TrueFFS of this explicitly. In this case, the customer can add a call to the **FL_IOCTL_DELETE_SECTORS** function after deleting the logical sectors.

- Notes:
1. This special handling is **not** necessary when using a FAT file system (sometimes also called a DOS file system), since the TrueFFS driver is automatically aware of space management.
 2. TrueFFS does not guarantee that the sector will actually be erased.

Input Record

```
typedef struct {  
    FLSDword firstSector;           /* First logical sector to delete */  
    FLSDword numberOfSectors; /* Number of sectors to delete */  
} flDeleteSectorsInput;
```

Output Record

```
typedef struct {  
    FLStatus status;  
} flOutputStatusRecord;
```

3.7. FL_IOCTL_FORMAT_PHYSICAL_DRIVE

This function formats a physical device by performing a combination of the following actions:

- Formats the device and partitions it into binary partitions and disk partitions.
- Protects up to two partitions of any kind.
- Writes the M-Systems x86 BIOS driver and firmware file (or simply leaves space for it in the first binary partition).
- Writes an empty FAT file system to disk partitions.

Notes: 1. Formatting destroys all existing data.

2. Some format scenarios leave all the disk partitions in the dismounted state, subsequently requiring a mount call.
3. INFTL/SAFTL-formatted devices support up to four partitions of any combination, binary and disk, providing there is at least one disk partition (NFTL-formatted devices support only one disk partition and one binary partition).
4. All protection keys must be inserted before calling this routine.
5. After placing OTW protection on a disk partition, the media can no longer be formatted. However a media with OTW protection on a binary partition can still be formatted using the leave binary partition flags (TL_LEAVE_SOME_BINARY_AREA / TL_LEAVE_BINARY_AREA).

Formatting is controlled by a set of parameters defined in a **FormatParams2** structure.

Input Record

```
typedef struct {  
    FLByte formatType; /* type of format as defined in blockdev.h */  
    FormatParams2 fp; /* Format parameters structure (defined in  
    flformat.h) */  
} flFormatPhysicalInput;
```

Output Record

```
typedef struct {  
    FLStatus status;  
} flOutputStatusRecord;
```

Formatting is controlled by a set of parameters defined in a **FormatParams2** structure and the flags given to the **formatType** field.

The **FormatParams2** structure is defined in **FLFORMAT.H** and contains the fields discussed in Section 3.7.1.

3.7.1. Translation Layer Formatting

<code>FLByte percentUse;</code>	NAND flash inherently contains some bad blocks. TrueFFS handles those blocks by managing a pool of spare blocks also called transfer units. This format parameter specifies the percentage of the media to use. According to the flash specifications, SLC flashes should use 98% while MLC flashes should use 95%. You can let TrueFFS automatically choose the percentage according to the flash spec by using the TL_DEFAULT_PERCENTAGE definition.
<code>FLByte noOfBDTLPartitions;</code>	Indicates the number of disk partitions (1-4). Note that NFTL-formatted devices must specify 1.
<code>FLByte noOfBinaryPartitions;</code>	Indicates the number of binary partitions (0-3) to create. If you need to keep some or all of your previous binary partitions, refer to the TL_LEAVE_BINARY_AREA and TL_LEAVE_SOME_BINARY_AREA flags of the irFlags field in the IOreq packet. Note that NFTL-formatted devices cannot specify more than one binary partition.
<code>BDTLPartitionFormatParams FAR1* BDTLPartitionInfo;</code>	Disk partition information record array (see definition below).
<code>BinaryPartitionFormatParams FAR1* binaryPartitionInfo;</code>	An array of binary partition information records (see definition below). This parameter is ignored if the TL_LEAVE_BINARY_AREA flag is set. However if the TL_LEAVE_SOME_BINARY_AREA is set as well, this array will describe only the new partitions.

3.7.2. Special Format Features

```
#ifdef WRITE_EXB_IMAGE
```

The EXB file (firmware file) can be written in one of two ways:

- A single large buffer, containing the entire EXB file, supplied to the format routine through the **exbBuffer** and **exbBufferLen** fields.
- Several consecutive calls to the **flPlaceExbByBuffer()** routine using buffers of any size (provided the first is at least 512 bytes long).

When using the **FL_IOCTL_PLACE_EXB_BY_BUFFER** extended function, you must ensure that the media is already formatted with a large enough binary sub-partition and the SPL signature. To format the media to contain a binary sub-partition with the SPL signature, either specify the exact binary sub-partition size using the **exbLen** field, or let the format routine determine the size automatically by setting **exbLen** to 0 and supplying at least 512 bytes of the EXB file (using the **exbBuffer** and the **exbBufferLen** fields). The format routine automatically detects the required size, formats the media with the required space and places the part of the EXB file that was already provided. The rest of the EXB buffer can be supplied using consecutive calls to the **FL_IOCTL_PLACE_EXB_BY_BUFFER** extended function.

<code>void FAR1* exbBuffer;</code>	Pointer to user buffer containing the EXB file. If no EXB should be written, use NULL.
<code>FLDword exbBufferLen;</code>	Size of the user buffer containing the EXB file.
<code>FLDword exbLen;</code>	Specify the media size to leave for the EXB file (a null (0) value will disable both the exbBuffer and exbBufferLen fields, and the automatic detection of the required size).
<code>FLWord exbWindow;</code>	Specify the DiskOnChip memory base address for the BIOS driver. (0 for automatic detection - the address where the IPL code resides).
<code>FLByte exbFlags;</code>	EXB-specific flags (see the flPlaceExbByBuffer() routine for the full flag list).

```
#endif /* WRITE_EXB_IMAGE */
```

<code>FLByte cascadedDeviceNo;</code>	Reserved.
<code>FLByte noOfCascadedDevices;</code>	Reserved.
<code>FLProgressCallback progressCallback;</code>	Pointer to user-defined progress callback routine. If no call back routine is necessary, set the
<code>/* FLStatus (*progressCallback)</code>	progressCallback field to NULL. The callback routine should return a status value; a value of flOK
<code>(FLWord totalUnitsToFormat, FLWord totalUnitsFormattedSoFar); */</code>	(0) allows formatting to continue, while any other value aborts the formatting process. TrueFFS calls this routine while it formats the media. The first argument

indicates the total number of units in the media, while the second indicates the currently formatted unit number.

If the **FL_REPORT_MOUNT_FEATURE** compilation flag is set, the format routine indicates the beginning of the mount of each of the device partitions by returning both call back routine arguments as 0. It also reports the progress of the mount routine. The first argument indicates the number of units in the partitions, and the second indicates the currently mounted unit. Note that a disk partition may contain more units than it exports.

3.7.3. Additional Special Features

The following fields are part of the extended function features, but are not used for DiskOnChip devices:

<code>FLDword vmAddressingLimit;</code>	Reserved for non-DiskOnChip devices.
<code>FLWord embeddedCISlength;</code>	Reserved for non-DiskOnChip devices.
<code>FLByte FAR1 * embeddedCIS;</code>	Reserved for non-DiskOnChip devices.

Both the disk and the binary partition parameters are passed through arrays of dedicated parameter records.

3.7.4. BDTL Partition Parameters

BDTL partition parameters are passed through the **BDTLPartitionInfo** field. This field is an array of **BDTLPartitionFormatParams** structures defined in **FLFORMAT.H** with the following fields:

<code>FLDword partitionSize;</code>	The size of the usable storage space, in bytes. The size is rounded upwards to a multiple of an erasable unit size. The size of the last partition is automatically calculated, and that parameter is ignored. Requesting a size smaller than an erasable unit for any partition, but the last will return an flBadParameters error code.
<code>unsigned noOfSpareUnits;</code>	Disk partitions need at least one spare erase unit to function as a write-able media and at least two to be fully protected against power failures. That unit is normally taken from the transfer units specified by the percentUsed field, but it is possible to specify additional units (which uses more media space). This ensures that if all the transfer units become bad and unerasable, the spare unit enables TrueFFS to maintain its read/write functionality. Conversely, if no spare units are available, the media may switch to read-only mode. The standard value is 3.

FLByte flag;	Any of the flags listed below.
	Note: this field name has been changed from TrueFFS 5.x. Its previous name was <code>signoffset</code> .
TL_NORMAL_FORMAT	Formats the media without placing any type of higher level format.
TL_FORMAT_FAT	Formats the media with the basic FAT format.
TL_OLD_FORMAT	Formats the media with basic FAT format, assuming one sector per cluster.
FLByte volumeId[4];	DOS partition identification number.
FLByte FAR1 * volumeLabel;	DOS partition label string. If NULL, no label.
FLByte noOfFATcopies;	It is customary to format DOS media with two FAT copies. The first copy is always used, but more copies make it possible to recover the FAT if it becomes corrupted (a rare occurrence). Conversely, this slows down performance and uses media space. The standard value is 2.
#ifdef HW_PROTECTION	
FLByte protectionKey[8];	The key for the protection.
FLByte protectionType;	A combination of the following flags:
PROTECTABLE	This partition can receive protection attributes.
READ_PROTECTED	Protects against read operations.
WRITE_PROTECTED	Protects against write operations.
LOCK_ENABLED	Enables the hardware lock signal.
CHANGEABLE_PROTECTION	This partition protection is changeable after formatting.
More information regarding the protection attributes can be found in the hardware protection section.	
#endif /* HW_PROTECTION */	

3.7.5. Binary Partition Parameters

Binary partition parameters are passed through the **binaryPartitionInfo** field. This field is an array of **BinaryPartitionFormatParams** structures defined in **FLFORMAT.H**, as follows:

<code>FLDword length;</code>	The size of the usable storage space, in bytes (not counting bad blocks). The size is rounded upwards to a multiple of an erasable unit size. Requesting a size smaller than an erasable unit for any binary partition returns an flBadParameters error code.
<code>FLByte sign[4];</code>	Signature of the binary partition to format. Valid values are 0x0000 0000 to 0xFFFF FFFE.
<code>FLByte flags;</code>	This field indicates one of the following partition modes:
<code>TL_NORMAL_FORMAT</code>	Normal binary partition, used for storing large OS image.
<code>TL_SPL_FORMAT</code>	Binary partition for small Secondary Program Loader (SPL). Partitions formatted with this mode may occupy double the requested capacity.
 <code>#ifndef HW_PROTECTION</code>	
<code>FLByte protectionKey[8];</code>	The key for the protection.
<code>FLByte protectionType;</code>	The same flags as in the disk partition structure above.
<code>#endif /* HW_PROTECTION */</code>	

3.7.6. Standard Formatting Parameters

You can use the **STD_FORMAT_PARAMS2** definition to initialize the **FormatParams2** record. These values reformat the media without modifying the binary partitions or the EXB file. A single unprotected disk partition is created in the remaining area of the volume, with two FAT copies, a volume ID of 0000, no volume label, two spare units and 98% usage. If the previous binary partition cannot be detected (unformatted media), no binary partition is created.

STD_FORMAT_PARAMS2 is defined in **FLFORMAT.H** with the following values:

<code>TL_DEFAULT_PERCENTAGE</code>	percentUse (98 for SLC or 95 for MLC)
<code>1,</code>	numberOfBDTLPartitions
<code>0,</code>	numberOfBinaryPartitions
<code>NULL</code>	BDTLPartitionInfo
<code>NULL,</code>	binaryPartitionInfo
 <code>#ifndef WRITE_EXB_IMAGE</code>	
<code>NULL,</code>	exbBuffer

```

0,                                exbBufLen
0,                                exbLen
0,                                windowBase
0,                                exbFlags
#endif /* WRITE_EXB_IMAGE */

0,                                cascadedDeviceNo.
0,                                noOfCascadedDevices
NULL,                             ProgressCallback
0x100001,                         vmAddressLimit for backwards compatibility
0,                                embeddedCISLength
NULL                               embeddedCIS

```

In addition to the above standard format parameters, you should initialize the **BDTLPartitionInfo** field describing the disk partition attributes (**BDTLPartitionFormatParams** record). Use the standard values for **STD_BDTL_PARAMS**, which are also defined in **FLFORMAT.H**.

```

0,                                partitionSize
3,                                noOfSpareUnits
TL_FORMAT_FAT ,                   flags
{0,0,0,0},                       volumeId[4]
NULL,                             volumeLabel
2,                                noOfFATcopies
#ifdef HW_PROTECTION
{0,0,0,0,0,0,0,0},               protectionKey[8]
0,                                protectionType
#endif /* HW_PROTECTION /

```

The **formatType** field can have one of the following values:

TL_NORMAL_FORMAT	Do not leave previous binary partitions.
TL_LEAVE_BINARY_AREA	Ignores all the binary partitions and firmware arguments of the function and leaves the data in those areas as is.
TL_SINGLE_FLOOR	Format only a selection of the floors.
TL_DO_NOT_PERFORM_DOWNLOAD	Do not issue a download operation at end of the format routine.

TL_LEAVE_SOME_BINARY_AREA

Ignores the firmware arguments, but allows you to leave only some of the previous binary partitions, while creating some new ones. The number of binary partition to leave is indicated by the **irLength** flag of the **IOreq** packet, while the **numberOfBinaryPartitions** field indicates the number of new partitions to create.

Note that this flag contains all of the **TL_LEAVE_BINARY_AREA** flag bits.

3.8. Hardware Protection

3.8.1. Overview

The extended functionality calls described in this section perform hardware read/write related operations, and consequently can be used only with DiskOnChip devices that have the required hardware support (Mobile DiskOnChip, DiskOnChip Millennium Plus, DiskOnChip G3/P3, and DiskOnChip 2000 G3 products).

There are different functions for handling binary and disk partitions. Their usage is identical, the only difference being that disk partitions require the **FL_IOCTL_BDTL_HW_PROTECTION** function while binary partitions require the **FL_IOCTL_BINARY_HW_PROTECTION** function.

3.8.2. Method of Operation

Mobile DiskOnChip, DiskOnChip Millennium Plus, DiskOnChip G3/P3, and DiskOnChip 2000 G3 products enable two partitions to be hardware-protected against read and/or write operations.

Due to the nature of the disk and binary partitions, read-protecting a partition in effect means that it is write-protected as well, since the TrueFFS algorithm must read from the flash in order to write to it.

Defining their size and protection attributes (read/write/changeable/lock) is done at the media formatting stage, using the DFORMAT utility or format extended function call. You may define one or two partitions as changeable, meaning that the password and attributes are fully configurable (read/write, both, none and vice versa) at any time. Note that unchangeable partition attributes (including the protection key) cannot be changed unless the media is reformatted.

Sticky Lock

Mobile DiskOnChip 128Mb, DiskOnChip Millennium Plus 16MB, DiskOnChip G3/P3, and DiskOnChip 2000 G3 products have an additional protection feature called the Sticky Lock. Once this feature is enabled, the protection key cannot be inserted until the device receives a hardware reset signal. When this mode is activated, it affects all protected partitions on the device provided the LOCK mechanism (LOCK_ENABLED) has been enabled.

OTW Protection

OTW is a special write-protection mode (in DiskOnChip G3/P3 products only). This attribute can be placed on an existing partition formatted as a protectable partition. Once a partition has an OTW-protected attribute, it is no longer possible to write to this partition (even using a protection key), and the partition actually becomes ROM-like.

Warning: If a disk partition becomes OTW-protected, it is no longer possible to reformat the device. (If it is a binary partition, reformatting is still possible using the leave partition flags of the format.)

Hardware LOCK Signal

A hardware signal can disable the use of the key for removal of the partition's protection. Although this is a hardware signal, it can be enabled and disabled per partition. Note that the **LOCK_ENABLED** flag is protected by the same key as the partition it protects.

Warning: If the hardware **LOCK** is asserted, and the **LOCK** signal is enabled using an IOCTL call, the signal cannot be disabled by software, even using the key. In this condition, DiskOnChip can no longer be formatted (until the signal is negated).

Note: The target volume does not have to be mounted before calling a hardware protection routine, and is not affected by the dismount process.

Each protected partition has its own unique attributes: key, read/write protection and the hardware LOCK signal enable state. TrueFFS exports several routines that enable changing these attributes, namely change key, change protection type (read/write/OTW protected), change hardware LOCK state (enabled or not), and Sticky Lock.

Changing any of these attributes causes a reset of the protection mechanism, and consequently, the removal of all the device protection keys. Care should be taken to avoid interference between different protected partitions. For example, a key inserted into one partition will be removed when another partition is instructed to change its protection attributes (such as by changing the key).

The only way to write or read to/from a read- or write-protected partition is to use the insert key call (not even reformatting will remove the protection). This is also true for modifying the partition attributes (key, read, write and Lock Enable state). The key is removed in each one of the following events:

- Power down.
- Change to one of the protection attributes (not necessarily to the same partition).
- Write operation to the IPL area using the **FL_IOCTL_WRITE_IPL** function.
- Removal of the protection key via the TrueFFS API.

- Notes:
1. In order to make a partition changeable, the specific flag must be added to the **protectionType** field in the format record. Without this flag, executing functions to change protection attributes will return error codes.
 2. For Mobile DiskOnChip and DiskOnChip Millennium Plus 32MB(256Mb) single die and 64MB(512Mb) products, only one partition per device can have changeable status.
 3. For Mobile DiskOnChip and DiskOnChip Millennium Plus 16MB(128Mb) and 32MB(256Mb) dual die, DiskOnChip G3/P3, and DiskOnChip 2000 G3 products, up to two partitions can be changeable.

3.8.3. FL_IOCTL_BDTL_HW_PROTECTION

The functions described in this section perform standard operations on partitions protected by the DiskOnChip hardware. Not all M-Systems devices support this feature. To find out if your DiskOnChip device supports this feature, call the **FL_IOCTL_INQUIRE_CAPABILITIES** function with the **SUPPORT_OTP_AREA** option (see Section 3.12). The hardware protection extended function is divided into sub-functions. All of the sub-functions use the same record for input and output.

Input Record

```
typedef struct {
    FLByte protectionType; /* See PROTECTION_GET_TYPE below */
    FLByte key[8];        /* The key */
    FLByte type;          /* One of the following flags: */
    #define PROTECTION_INSERT_KEY    - 0 - Insert key (disables the
    protection)
    #define PROTECTION_REMOVE_KEY    - 1 - Remove key
                                         (restores the protection)
    #define PROTECTION_GET_TYPE      - 2 - Do not enable the H/W LOCK pin
    #define PROTECTION_DISABLE_LOCK - 3 - Enable the H/W LOCK pin
    #define PROTECTION_ENABLE_LOCK  - 4 - Returns the protection status
    #define PROTECTION_CHANGE_KEY   - 5 - Change the protection key
    #define PROTECTION_CHANGE_TYPE  - 6 - Change the protection type
                                         (read \ write protected)

} flProtectionInput;
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

Note: Some protection operations return values in the **flProtectionInput** record.

Following are the action types and required arguments in the **flProtectionInput** record:

PROTECTION_INSERT_KEY: Inserts the key to a protected area.

<i>flProtectionInput</i> Parameters	
Key	The key to be inserted.

Note: Inserting a wrong key to a partition that already has a key inserted does not fail.

PROTECTION_REMOVE_KEY: Removes the key from a protected partition.

PROTECTION_DISABLE_LOCK: Disables the DiskOnChip LOCK pin signal effect on the key.

PROTECTION_ENABLE_LOCK: Enables the DiskOnChip LOCK pin signal effect on the key.

PROTECTION_CHANGE_KEY: Change the key of a protected partition.

<i>flProtectionInput</i> Parameters	
Key	The new key for the protection area.

CHANGE_PROTECTION_TYPE: Changes the protection type.

<i>flProtectionInput</i> Parameters			
protectionType	PROTECTABLE	1	Must be added for the operation to succeed.
	READ_PROTECTED	2	Partition is protected against read operations.
	WRITE_PROTECTED	4	Partition is protected against write operations.

PROTECTION_GET_TYPE: Gets a protected partition status.

<i>flProtectionInput</i> Parameters			
Type	Action type.		
protectionType	PROTECTABLE	1	Partition can be protected.
	READ_PROTECTED	2	Partition is protected against read operations.
	WRITE_PROTECTED	4	Partition is protected against write operations.
	LOCK_ENABLED	8	Hardware LOCK signal is enabled.
	LOCK_ASSERTED	16	Hardware LOCK signal is currently asserted.
	KEY_INSERTED	32	Protection is temporarily removed.
	CHANGEABLE_PROTECTION	64	Partition protection attributes can be changed without fully reformatting the media.
	OTW_PROTECTED	128	The key can never be inserted to this partition.
	STICKY_LOCK_ASSERTED	256	The key cannot be inserted to this partition until the next hardware reset.
Returns			
Status	f1OK: Success. f1NotProtected: Not a protected partition. f1FeatureNotSupported: The driver was not compiled with the required feature or when the devices does not support it.		

3.8.4. FL_IOCTL_BINARY_HW_PROTECTION

This extended function is identical to the **FL_IOCTL_BDTL_HW_PROTECTION** function, but is used exclusively for binary partitions.

3.9. FL_IOCTL_OTP

The functions described in this section perform standard operations on the One Time Programmable (OTP) area. Not all M-Systems devices support this feature. To determine if your DiskOnChip supports the OTP area, call the **FL_IOCTL_INQUIRE_CAPABILITIES** function with the **SUPPORT_OTP_AREA** option (see Section 3.12).

Mobile DiskOnChip, DiskOnChip Millennium Plus, and all SAFTL-based devices have a ROM-like hardware feature called an OTP area. This feature provides a dedicated area on the flash that is written to once and then locked permanently by the DiskOnChip hardware. After the first write to the OTP area, (EDC is automatically added) it is hardware-protected against all write and erase operations. The total size of the area, the actual used size and the locked state can be retrieved in addition to performing normal read operations of the area.

The OTP extended function is divided into sub-functions. A pointer to the **flOtpInput** structure is passed to all of the OTP sub-functions. The same record is sent both as input and as output.

Input Record

```
typedef struct {
    FLDDword      length;      /* Length to read/write/size in bytes */
    FLDDword      usedSize;    /* The written area size in bytes      */
    FLByte        lockedFlag; /* The area condition (LOCKED_OTP)    */
    FLByte FAR*   buffer;      /* pointer to user buffer              */
    FLWord        type;        /* One of the types bellow            */
} flOtpInput;                /* Is used for output as well */

#define OTP_SIZE      1 - Get OTP statistics.
#define OTP_READ      2 - Read from the OTP area.
#define OTP_WRITE_LOCK 3 - Write and permanently lock the OTP
area.
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

OTP_SIZE: Gets the size of the OTP area.

<i>flotpInput</i> Parameters	
Type	OTP_SIZE
Returns	
Status	<i>flOK</i> : Success.
Length	The length of the OTP area, in bytes
usedSize	The used size of the OTP area, in bytes
lockedFlag	The area current state: #define LOCKED_OTP 1: The area is currently locked.

OTP_READ: Reads from the OTP area to a user buffer.

<i>flotpInput</i> Parameters	
Type	OTP_READ
Length	The length to read, in bytes.
usedSize	The offset of the first byte to read.
Buffer	Pointer to the user buffer to be read to.
Returns	
Status	<i>flOK</i> : Success. <i>flDataError</i> : EDC/ECC error. <i>flBadLength</i> : Size exceeds OTP area size. <i>flFeatureNotSupported</i> : The driver was not compiled with the required feature or when the devices does not support it.

OTP_WRITE_LOCK: Write to the OTP area, add EDC/ECC and lock the OTP area.

<i>flotpInput</i> Parameters	
type	OTP_WRITE_LOCK
length	The length to write, in bytes.
buffer	Pointer to the user buffer to write from.
Returns	
status	<i>flOK</i> : Success <i>flDataError</i> : EDC/ECC error. <i>flHWPProtection</i> : OTP was already locked. <i>flBadLength</i> : Size exceeds the OTP area size. <i>flFeatureNotSupported</i> : The driver was not compiled with the required feature or when the devices does not support it.

3.10. Unique ID (UID)

The functions described in this section perform standard operations on the ID region of DiskOnChip.

Each Mobile DiskOnChip, DiskOnChip Millennium Plus, and SAFTL-formatted device has a unique 16-byte ID number, called a Unique ID (UID). This number is randomly generated and guaranteed to be unique to this DiskOnChip device, meaning that no two units are the same. When ordering large quantities of DiskOnChip, a 4-byte customer ID signature can be burned into the units (at the FAB stage). Both IDs are hardware protected against write and erase operations.

3.10.1. FL_IOCTL_CUSTOMER_ID

Returns the hardware-embedded 4-byte customer ID information.

Input Record

DO NOT CARE

Output Record

```
{
    FLByte          id[4] ;
    FLStatus        status;
} flCustomerIdOutput;
```

3.10.2. FL_IOCTL_UNIQUE_ID

Returns the hardware-embedded 16-byte UID information.

Input Record

DO NOT CARE

Output Record

```
{
    FLByte          id[16];
    FLStatus        status;
} flUniqueIdOutput;
```

3.11. FL_IOCTL_NUMBER_OF_PARTITIONS

Returns the number of disk partitions in a specified device.

Input Record

DO NOT CARE

Output Record

```
typedef struct {  
    FLByte    noOfPartitions;  
    FLStatus   status;  
} flCountPartitionsOutput;
```

3.12. FL_IOCTL_INQUIRE_CAPABILITIES

Returns confirmation as to whether or not the current hardware and software support a specific feature.

Input Record

```
typedef struct {
    FLCapability    capability;    /* See flags below */
    SUPPORT_UNERASABLE_BBT        - 2
    SUPPORT_MULTIPLE_BDTL_PARTITIONS - 3
    SUPPORT_MULTIPLE_BINARY_PARTITIONS - 4
    SUPPORT_HW_PROTECTION        - 5
    SUPPORT_HW_LOCK_KEY          - 6
    SUPPORT_CUSTOMER_ID          - 7
    SUPPORT_UNIQUE_ID            - 8
    SUPPORT_DEEP_POWER_DOWN_MODE - 9
    SUPPORT_OTP_AREA             - 10
    SUPPORT_WRITE_IPL_ROUTINE    - 11
    SUPPORT_OTW_PROTECTION       - 12
    SUPPORT_STICKY_LOCK_PROTECTION - 13
    SUPPORT_RUGGED_FILE_SYSTEM   - 14
} flCapabilityInput;
```

Output Record

```
typedef struct {
    FLCapability    capability; /* See flags bellow */
    CAPABILITY_NOT_SUPPORTED    - 0
    CAPABILITY_SUPPORTED        - 1
    FLStatus status;
} flOutputStatusRecord;
```

3.13. FL_IOCTL_SET_ENVIRONMENT_VARIABLES

This function is no longer supported from TrueFFS 5.1 and up. Please see Section 3.18 for a description of the replacement function `FL_IOCTL_EXTENDED_ENVIRONMENT_VARIABLES`.

3.14. FL_IOCTL_PLACE_EXB_BY_BUFFER

Places an EXB (firmware) file on the media using small buffers.

- Notes:
1. The first buffer must be at least 512 bytes long.
 2. Binary operations to binary partition 0 (except for remove and insert key) will reset the process.
 3. Only M-Systems EXB files are supported by this routine.
 4. Calling this routine with a partition number other than 0 returns an **flBadDriveHandle** error code.

Input Record

```
typedef struct {
    FLByte  FAR1*  buf;           /* Buffer of EXB file           */
    FLWord   bufLen;             /* Buffer length                 */
    FLByte   exbFlags           /* A combination of flags see below: */
    FLWord   exbWindow          /* Explicitly set device window.   */
                                /* 0 will automatically set window */
} flPlaceExbInput;

exbFlags  INSTALL_FIRST  1    Makes the device the first hard drive.
          QUIET          4    Do not show titles while BIOS expansion is found.
          INT15_DISABLE  8    Disables INT15 hooking.
          FLOPPY        16   Makes the device assume drive [A:] (does not make it
                                bootable).
          SIS5598        32   Supports Windows NT platforms with the SIS5598 VGA
                                chipset.
          EBDA_SUPPORT   64   Supports BIOS with EBDA.
          NO_PNP_HEADER  128  Do not place the PNP header.
          LEAVE_EMPTY    256  Leaves the firmware area empty.
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

3.15. FL_IOCTL_WRITE_IPL

This function is no longer supported from TrueFFS 5.1 and up. Please see Section 3.20 for a description of the replacement function **FL_IOCTL_EXTENDED_WRITE_IPL**.

3.16. FL_IOCTL_DEEP_POWER_DOWN_MODE

Changes the power consumption mode of Mobile DiskOnChip, DiskOnChip Millennium Plus, or SAFTL-formatted devices. To verify that your device supports this feature, call the **FL_IOCTL_INQUIRE_CAPABILITIES** function with the **SUPPORT_DEEP_POWER_DOWN_MODE** option (see Section 3.12).

Note: If your platform uses DiskOnChip as the boot device, and your system initiates a reset command without waking up DiskOnChip from Deep Power Down mode, the platform will not be able to boot. This is also applicable for x86 architecture when DiskOnChip is used as a flash disk with a BIOS driver.

Input Record

```
typedef struct {
    FLByte state; /* DEEP_POWER_DOWN    - low power consumption    */
                  /* otherwise          - regular power consumption */
} flPowerDownInput;

#define DEEP_POWER_DOWN    1
#define EXIT_DEEP_POWER_DOWN 0
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

3.17. FL_IOCTL_BDK_OPERATION

The functions described in this section perform standard operations on a binary partition (read/write/erase/create/get size of partition). Binary partitions cannot be accessed by the file system, and are reserved for customer use. The most common use for a binary partition is to store system boot code or the OS image file. Operations on binary partition blocks do not affect file system activity and will not slow down performance. For a full description, see the *DiskOnChip Boot Software Development Kit (BDK)* developer guide.

The BDK extended functions are divided into sub-functions.

Input Record

```
typedef struct {
    FLByte type; /* One of the operation types mentioned bellow: */
    BDKStruct bdkStruct; /* Specific parameters see bellow */
} flBDKOperationInput;

#define BDK_INIT_READ    0
#define BDK_READ         1
#define BDK_INIT_WRITE   2
#define BDK_WRITE        3
#define BDK_ERASE        4
#define BDK_CREATE       5
#define BDK_GET_INFO     6
```

The **bdkStruct** record is common to all of the sub-functions and is defined as follows:

```
typedef struct {
    FLByte oldSign[4]; /* Signature of the Binary partition to work on */
    FLByte newSign[4]; /* Signature of the new Binary partition to */
                        /* create (relevant only for bdkCreate) */
    FLByte signOffset; /* Offset of the signature - must be set to 8 */
    FLDword startingBlock; /* Number of blocks to skip */
    FLDword length; /* Number of bytes to read/write or number of */
                  /* blocks to erase */
    FLByte flags; /* Option flags: Activate EDC/ECC mechanism, */
                /* */
    FLByte FAR1 * bdkBuffer; /* Pointer to Read/write buffer */
} BDKStruct;
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

Note: Some binary operations return values in the **bdkStruct** input record.

3.17.1. BDK_INIT_READ

Performs an initialization procedure on the binary partition before **BDK_READ** is called.

This function checks that the read operation about to be performed on the binary partition is within the sub-partition's "already written" boundary. This function must be called before any **BDK_READ** operation, and is followed by a sequence of **BDK_READ** calls. If the EDC flag is on, the error detection and correction (EDC/ECC) mechanism is activated.

<i>BDKStruct</i> Parameters	
startingBlock	Unit number from which to start the read operation (counting from zero).
length	Number of bytes to read.
oldSign	Signature of subpartition.
flags	EDC – not relevant for SAFTL formatted devices.
signOffset	Must be set to 8.
Return	
FLStatus	0 on success, non-zero on failure.

3.17.2. BDK_READ

Reads from the sub-partition of a binary partition.

BDK_INIT_READ must be called immediately before this operation. It is recommended to keep the **length** parameter at a flash page size (Currently 512 – 2048) and to use full erasable blocks when possible.

BDKStruct Parameters	
length	Number of bytes to read
bdkBuffer	Pointer to a buffer that receives the binary data
Return	
FLStatus	0 on success, non-zero on failure

3.17.3. BDK_INIT_WRITE

Performs an initialization procedure before **BDK_WRITE** is called.

This function checks that the write operation about to be performed on the binary partition is within the sub-partition boundary. **BDK_INIT_WRITE** must be called before calling **BDK_WRITE**.

BDKStruct Parameters	
startingBlock	Unit number from which to start the write operation.
length	Number of bytes to write.
oldSign	Signature of the subpartition.
flags	BDK_COMPLETE_IMAGE_UPDATE – Marks the last block with "FFF", enabling TrueFFS to identify the end of the used sub-partition.
signOffset	Must be set to 8.
Return	
FLStatus	0 on success, non-zero on failure.

3.17.4. BDK_WRITE

Writes to the sub-partition of a binary partition.

BDK_INIT_WRITE must be called immediately before this operation. It is recommended to keep the **length** parameter at a flash page size (Currently 512 – 2048) and to use full erasable blocks when possible.

BDKStruct Parameters	
length	Number of bytes to write.
oldSign	Signature of the subpartition to be written to.
bdkBuffer	Pointer to a buffer containing the binary data to write.
Return	
FLStatus	0 on success, non-zero on failure.

3.17.5. BDK_ERASE

Erases sequential blocks in the subpartition of a binary partition. The data in these blocks is erased, and only the signature indicating the sub-partition remains.

BDKStruct Parameters	
startingBlock	Block number of the sub-partition where the erase operation starts.
length	Number of blocks to erase.
oldSign	Signature of the sub-partition to be erased.
signOffset	Must be set to 8.
Return	
FLStatus	0 on success, non-zero on failure.

3.17.6. BDK_CREATE

Creates a new, empty sub-partition in a binary partition by overwriting an existing sub-partition. A new sub-partition is always created at the end of an old sub-partition. If the new sub-partition is larger than the old sub-partition, an error status is returned.

For example, if you start with a 4MB sub-partition with signature “AAAA” and create a 1MB subpartition with signature “BBBB” over the old sub-partition, the result is a 3MB sub-partition with signature “AAAA” followed by a 1MB sub-partition with signature “BBBB”.

- Notes:
1. The value 0xFFFF FFFF is not a valid binary partition signature, and should not be used.
 2. If a sub-partition with the same signature already exists, it might be hard to determine the original from the new sub-partition. To avoid complications, save the data from the old sub-partition and rewrite it after creating the new “enlarged” partition.
 3. The **bdkCreate** function (like any other binary partition function) cannot cross the binary partition boundaries.

BDKStruct Parameters	
length	Length of the sub-partition to be created, in erasable blocks.
oldSign	Signature of the existing sub-partition.
newSign	Signature of the sub-partition to create.
signOffset	Must be set to 8.
Return	
FLStatus	0 on success, non-zero on failure.

3.17.7. BDK_GET_INFO

Returns the number of binary partitions on the physical drive, the total size of a specific sub-partition and its used size.

<i>BDKStruct</i> Parameters	
oldSign	Signature of the sub-partition to determine its length.
signOffset	Must be set to 8.
startingBlock	Block of the sub-partition where the search starts.
Return	
FLStatus	0 on success, non-zero on failure.
<i>Ioreq</i> Parameter	
irLength	Binary partition physical length, in bytes.
<i>BDKStruct</i> Returns	
length	Used virtual size of the sub-partition, in bytes.
startingBlock	Virtual size of the sub-partition, in bytes.
flags	Number of binary partitions.

3.18. FL_IOCTL_EXTENDED_ENVIRONMENT_VARIABLES

TrueFFS-based drivers support several run-time configuration variables. For each option, there is a global variable inside the driver that dictates the driver behavior. These global variables are called environment variables. Most drivers already include various mechanisms for setting the values of the environment variables (such as the registry for Windows CE). If your driver does not support a dedicated mechanism, use this extended function to customize your application.

Input Record

```
typedef struct {
    FLEnvVars    varName;    /* Variable Name */
    FLDword      varValue;   /* Variable value */
    FLDword      flags;      /* Either of the following flags */
        FL_APPLY_TO_ALL      - Change all partitions and sockets
        FL_APPLY_TO_SOCKET  - Change a specific socket
        FL_APPLY_TO_VOLUME  - Change a specific partition
    } flExtendedEnvVarsInput;
```

Table 3: Run-time Customization Environment Variables

Variable Name (varName)	Flags	Variable Value (varValue)	Comments
FL_TL_CACHE_ENABLED	FL_APPLY_TO_ALL	FL_OFF FL_ON	Enabling this option improves performance, but requires additional RAM resources. (NFTL, INFTL and SAFTL use a small part of each flash unit and sector for control information that allows accessing the data stored on DiskOnChip as a Virtual Block Device.) If <code>flUseNFTLCache</code> is set to <code>FL_ON</code>, the TL keeps an identical table of the necessary control information in RAM. Whenever it is necessary to change a table's entry or a unit's flags, the TL first reads the data and then updates the table both in RAM and on DiskOnChip. If the TL reads the table entry from RAM, it saves time by not having to read that control data from DiskOnChip. Note that access to this control data is required for every read/write API function.

Variable Name (varName)	Flags	Variable Value (varValue)	Comments
			In SAFTL, this variable has no affect.
FL_DOC_8BIT_ACCESS	FL_APPLY_TO_ALL	FL_OFF FL_ON	Defines the type of access used by the access layer when accessing the DiskOnChip memory window. Possible values: FL_ON - The access layer routines use the <i>flmemxxx</i> routines, which may be slower, but are less prone to customization problems. FL_OFF - The access layer routines use the <i>flxxx</i> routines, which may use larger operands and even assembler code (<i>memcpy</i> library function). These routines are expected to be more efficient (i.e. faster), in some boards and CPUs, but may not work with other CPUs or buses.
FL_SUSPEND_MODE	FL_APPLY_TO_ALL	[FL_OFF] FL_SUSPEND_WRITE FL_SUSPEND_IO	When set to FL_SUSPEND_WRITE no write or erase operations can be performed. When set to FL_SUSPEND_IO no read/write/erase operations are allowed. Use when a power down event is expected.
FL_MTD_BUS_ACCESS_TYPE	FL_APPLY_TO_ALL FL_APPLY_TO_SOCKET	[FL_BUS_HAS_8BIT_ACCESS] Bus can access 8-bit [FL_BUS_HAS_16BIT_ACCESS] Bus can access 16-bit [FL_BUS_HAS_32BIT_ACCESS] Bus can access 32-bit [FL_NO_ADDR_SHIFT] No address shift FL_SINGLE_ADDR_SHIFT Single address shift FL_DOUBLE_ADDR_SHIFT	Specify the type of access your platform has to the DiskOnChip memory window. If your platform allows more then one type of access, specify them all.

Variable Name (varName)	Flags	Variable Value (varValue)	Comments
		Double address shift	
FL_VERIFY_WRITE_BDTL	FL_APPLY_TO_ALL, FL_APPLY_TO_SOCKET, FL_APPLY_TO_VOLUME	FL_UPS FL_OFF FL_ON Defaults: [NFTL - FL_UPS] [INFTL - FL_OFF] [SAFTL - FL_OFF]	Selects the verify write operation mode for disk partitions: FL_UPS : Verify write option is disabled (NFTL default). FL_OFF : Full protection against power failure in the block device level (INFTL/SAFTL default). FL_ON : Perform verification of every write operation.
FL_VERIFY_WRITE_BINARY	FL_APPLY_TO_ALL FL_APPLY_TO_SOCKET FL_APPLY_TO_VOLUME	[FL_OFF] FL_ON	Selects the verify write operation mode for binary partitions: FL_OFF : Verify write option is disabled. FL_ON : Perform read after every write operation.
FL_VERIFY_WRITE_OTHER	FL_APPLY_TO_ALL FL_APPLY_TO_SOCKET	FL_OFF [FL_ON]	Selects the verify write operation mode for non-partition related (OTP area, media header, etc) operations: FL_OFF : Verify write option is disabled. FL_ON : Perform read after every write operation.
FL_MAX_CACHE_SIZE	FL_APPLY_TO_ALL FL_APPLY_TO_SOCKET FL_APPLY_TO_VOLUME	FLDword 1-0xFFFF [Default is 64 or 0xFFFF]	Sets the size of the SAFTL cache size. If TL_CACHE is enabled and flUseNFTLCache is set to FL_ON , the default changes to 0xffff, allowing the TL to allocate the exact number of units needed to cache the entire disk partition. This variable has no affect on NFTL- and INFTL-formatted devices.
FL_DO_NOT_WAIT_FOR_READY	FL_APPLY_TO_ALL FL_APPLY_TO_SOCKET FL_APPLY_TO_VOLUME	[FL_OFF] FL_ON	Skips the time spent in waiting for the flash ready signal before returning to the user. In this mode, the last write operation is checked for success only on the next TrueFFS flash access.

Output Record

```
typedef struct {  
    FLDword  prevValue; /* The previous value of the variable */  
    FLStatus status;  
} flExtendedEnvVarsOutput;
```

3.19. FL_IOCTL_VERIFY_VOLUME

Scans the partition for sectors that are only partially written due to power failure events.

Input Record

```
typedef struct {  
    FLDword    flags;      /* Must be set to 0 */  
} flVerifyVolumeInput;
```

Output Record

```
typedef struct {  
    void FAR1*  callBack;  /* Must be set to null */  
    FLStatus    status;  
} flVerifyVolumeOutput;
```

3.20. FL_IOCTL_EXTENDED_WRITE_IPL

Writes data to the IPL region of DiskOnChip Millennium Plus, Mobile DiskOnChip, and SAFTL-formatted devices.

- Notes:
1. The IPL for DiskOnChip 2000 is stored on a ROM block and cannot be changed.
 2. The 512-byte SRAM block in DiskOnChip Millennium products is initialized with the contents of the first 512-byte page of unit 0 on the flash media. The following page should hold a redundant copy of the IPL code, which is read automatically of there is an EDC error on the first page. Both pages can be written using the binary partition extended functions, provided that the media is preformatted with a binary partition.

To determine if your device supports write IPL extended functionality, call the **FL_IOCTL_INQUIRE_CAPABILITIES** function with the **SUPPORT_WRITE_IPL_ROUTINE** option (see Section 3.12).

The available flags are:

<code>FL_IPL_MODE_NORMAL</code>	0	No special mode.
<code>FL_IPL_DOWNLOAD</code>	1	Force download of a new IPL.
<code>FL_IPL_MODE_XSCALE</code>	4	Enable XScale mode: Full asynchronous address change is detected automatically by the DiskOnChip controller.

Note: This flag is relevant only for the DiskOnChip Millennium Plus 16MB and Mobile DiskOnChip 16MB. Older devices do not support this feature while newer devices incorporate it automatically.

Input Record

```
typedef struct {
    FLByte FAR* buf;      /* IPL data buffer          */
    FLWord      flags;    /* IPL flags                */
    FLWord      bufLen;   /* IPL data buffer length  */
} flIplInput;
```

Output Record

```
typedef struct {
    FLStatus status;
} flOutputStatusRecord;
```

3.21. FL_IOCTL_IS_QUICK_MOUNT_VALID

Queries the validity of the quick mount information.

Input Record

DO NOT CARE.

Output Record

```
typedef struct {  
    FLByte          mode; /* Either FL_ON for valid and FL_OFF  
    for not */  
    FLStatus        status;  
} flGetQuickMountOutput;
```

3.22. FL_IOCTL_WRITE_QUICK_MOUNT_INFO

Dumps the current RAM tables to the flash. This extended function must be called after the media has been mounted.

Input Record

DO NOT CARE.

Output Record

DO NOT CARE.

3.23. FL_IOCTL_CLEAR_QUICK_MOUNT_INFO

Discards the current quick mount, insuring a full mount operation.

Input Record

DO NOT CARE.

Output Record

DO NOT CARE.

3.24. FL_IOCTL_COMPLETE_OPERATION

Ensures that the last write operation was completed successfully. This extended function is not required for driver operation, and is relevant only when working in Do Not Wait For Ready mode and you want to make sure the last operation was actually completed.

Do Not Wait For Ready mode enables the TrueFFS driver to return to the user while the flash is still busy with a long write or erase operation. This mode may improve the overall write performance when writing to small files, as it saves the software overhead between flash write operations.

Input Record

DO NOT CARE.

Output Record

DO NOT CARE.

4. ADDITIONAL INFORMATION

Additional information about DiskOnChip, including application notes, data sheets, and utilities can be found at <http://www.m-sys.com>.

Additional tools and documents are listed in the following table:

Document/Tool	Description
Developer Guide	DiskOnChip Boot Software Development Kit 6.x (BDK)
Installation Manual	Manuals for specific OSs are enclosed with the driver package in PDF or text format
User Manual	DiskOnChip Software Utilities for TrueFFS 6.x

HOW TO CONTACT US

USA

M-Systems Inc.
8371 Central Ave, Suite A
Newark CA 94560
Phone: +1-510-494-2090
Fax: +1-510-494-5545

Japan

M-Systems Japan Inc.
Asahi Seimei Gotanda Bldg., 3F
5-25-16 Higashi-Gotanda
Shinagawa-ku Tokyo, 141-0022
Phone: +81-3-5423-8101
Fax: +81-3-5423-8102

Taiwan

M-Systems Asia Ltd.
Room B, 13 F, No. 133 Sec. 3
Min Sheng East Road
Taipei, Taiwan
R.O.C.
Tel: +886-2-8770-6226
Fax: +886-2-8770-6295

China

M-Systems China Ltd.
Room 121-122
Bldg. 2, International Commerce & Exhibition Ctr.
Hong Hua Rd.
Futian Free Trade Zone
Shenzhen, China
Phone: +86-755-8348-5218
Fax: +86-755-8348-5418

Europe

M-Systems Ltd.
7 Atir Yeda St.
Kfar Saba 44425, Israel
Tel: +972-9-764-5000
Fax: +972-3-548-8666

Internet

<http://www.m-sys.com>

General Information

info@m-sys.com

Sales and Technical Information

techsupport@m-sys.com

This document is for information use only and is subject to change without prior notice. M-Systems Flash Disk Pioneers Ltd. assumes no responsibility for any errors that may appear in this document. No part of this document may be reproduced, transmitted, transcribed, stored in a retrievable manner or translated into any language or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without prior written consent of M-Systems.

M-Systems products are not warranted to operate without failure. Accordingly, in any use of the Product in life support systems or other applications where failure could cause injury or loss of life, the Product should only be incorporated in systems designed with appropriate and sufficient redundancy or backup features.

Contact your local M-Systems sales office or distributor, or visit our website at www.m-sys.com to obtain the latest specifications before placing your order.

©2003 M-Systems Flash Disk Pioneers Ltd. All rights reserved.

M-Systems, DiskOnChip, DiskOnChip Millennium, DiskOnKey, DiskOnKey MyKey, FFD, Fly-By, iDiskOnChip, iDOC, mDiskOnChip, mDOC, Mobile DiskOnChip, Smart DiskOnKey, SuperMAP, TrueFFS, uDiskOnChip and uDOC are trademarks or registered trademarks of M-Systems Flash Disk Pioneers, Ltd. Other product names or service marks mentioned herein may be trademarks or registered trademarks of their respective owners and are hereby acknowledged. All specifications are subject to change without prior notice.